Creare video e contenuti multimediali

senza strumenti a pagamento

Guida pratica, passo per passo

Otto strumenti gratuiti e una tecnica: come far lavorare un agente sui propri profili, e come montare tutto con ffmpeg.

Con esempi, comandi e gli avvisi su ciò che è gratuito e ciò che non lo è.

Perché questa guida

Il multimedia si può fare senza abbonamenti. Non è una promessa da volantino: è che negli ultimi anni gli strumenti **aperti** hanno raggiunto quelli commerciali, e in alcuni casi li hanno superati — perché chi costruisce un prodotto chiuso deve proteggere il proprio margine, mentre chi costruisce uno strumento aperto può permettersi di essere usato da chiunque, ovunque, senza chiedere il conto.

Questa guida mette in fila otto strumenti e **una tecnica**. Gli strumenti si scelgono in base a cosa devi fare; la tecnica, invece, cambia il modo in cui lavori, ed è per questo che le ho dato il capitolo più lungo.

> *Il criterio con cui ho scritto: per ogni strumento dico **cosa fa**, **quanto costa davvero** e **cosa serve per usarlo**. Dove un prodotto è a pagamento lo dico apertamente: non serve a nessuno un elenco che promette gratuità e poi chiede la carta di credito al primo salvataggio.*

Il quadro d'insieme: la catena del lavoro

Prima degli strumenti, conviene vedere la filiera. Un video, un tutorial, una presentazione multimediale nascono sempre da cinque passaggi:

1. **Il materiale** — quello che si vede: una registrazione dello schermo, una pagina web, un'immagine, un oggetto 3D.
2. **Il montaggio** — tagliare, unire, dissolvenze, formato.
3. **La voce** — il parlato che accompagna.
4. **Il ritmo e l'animazione** — movimenti, comparse, sottotitoli.
5. **La pubblicazione** — il formato giusto per il posto giusto.

Ogni strumento di questa guida lavora su uno o più di questi passaggi. Alla base di tutti, però, c'è **una cosa sola**: `ffmpeg`. È il tornio su cui girano anche i programmi con l'interfaccia grafica. Capirlo è il modo più veloce per smettere di dipendere da un'applicazione specifica.

1 · FFmpeg: la base di tutto

`ffmpeg` è un programma a riga di comando che legge, trasforma e scrive audio e video. Non ha finestre, non ha pulsanti: gli si dice cosa fare e lo fa. È **libero e gratuito** (licenza LGPL 2.1 o superiore; alcune parti opzionali come il codificatore `libx264` sono GPL 2, e chi le include porta l'insieme sotto GPL), gira su Windows, Mac e Linux, e — questa è la parte che conta — **quasi tutti gli strumenti con l'interfaccia grafica lo usano dentro**: quando un programma "esporta in MP4", nella maggior parte dei casi sta chiamando ffmpeg.

> *Un dettaglio che rassicura: per ffmpeg **non esiste una versione a pagamento**, nemmeno volendo. Non c'è un piano "pro": c'è il programma, e la licenza libera. È il motivo per cui vale la pena investirci tempo.*

Imparare cinque comandi di ffmpeg vale più che imparare cinque programmi.

Cosa sa fare, in concreto

- **Convertire** un formato in un altro (`.mov` → `.mp4`, `.webm` → `.mp4`).
- **Ritagliare** un pezzo di video (dal secondo 12 al secondo 30).
- **Unire** più video in uno solo.
- **Cambiare formato e risoluzione** (portare tutto a 1920×1080, il formato 16:9).
- **Estrarre l'audio** da un video, o **sostituirlo**.
- **Aggiungere una traccia musicale** e abbassarla automaticamente quando qualcuno parla.
- **Dissolvenze** fra una scena e l'altra.
- **Registrare lo schermo** e la **voce del computer**.

I cinque comandi che servono davvero

Convertire e portare tutto al formato di pubblicazione (16:9, 1920×1080):

```
ffmpeg -i sorgente.mov -vf "scale=1920:1080" -c:v libx264 -crf 20 -pix_fmt yuv420p uscita.mp4
```

Ritagliare un pezzo (dal minuto 0:12 per 18 secondi):

```
ffmpeg -ss 12 -t 18 -i video.mp4 -c copy pezzo.mp4
```

Unire due video (con un file di elenco `lista.txt` che contiene le righe `file 'uno.mp4'`):

```
ffmpeg -f concat -safe 0 -i lista.txt -c copy unito.mp4
```

Estrarre l'audio:

```
ffmpeg -i video.mp4 -vn -c:a copy audio.m4a
```

Registrare lo schermo su Windows, **con il cursore del mouse**:

```
ffmpeg -f gdigrab -framerate 15 -draw_mouse 1 -i desktop schermo.mp4
```

Le tre trappole

1. `-crf` **è la qualità.** Più basso è il numero, migliore è l'immagine e più grande il file. `18` è quasi senza perdita, `23` è il compromesso abituale, `28` è già compresso.
2. **La larghezza e l'altezza devono essere pari.** Un video largo 1921 pixel fa fallire la codifica con un errore poco chiaro: si arrotonda a 1920.
3. **Il cursore del mouse non c'è, a meno che non lo si chieda.** Nella registrazione dello schermo, senza `-draw_mouse 1` il video esce senza puntatore, e non c'è modo di aggiungerlo dopo.

2 · Recordly: registrare lo schermo

Un registratore di schermo serve a una cosa sola, ma indispensabile: **mostrare quello che fai**. È lo strumento con cui nascono i tutorial.

Recordly (sito **recordly.dev**) è un registratore di schermo con un editor incorporato: ingrandimento automatico sulle zone che contano, movimento morbido del cursore, bolla della webcam, linea del tempo, esportazione in MP4 o GIF. È **gratuito e open source**, con licenza **AGPL-3.0**: si può usare, studiare e modificare; chi ne distribuisce una versione modificata deve renderne pubblico il sorgente. Gira **in locale**, su macOS 14 o superiore, Windows 10 (build 19041 o superiore) e Linux.

> ***Attenzione a un tranello.*** *Cercando "Recordly" si incontra anche* ***recordly.vip****, che vende un prodotto* ***diverso e a pagamento*** *(circa 20 dollari una volta sola) e* ***non è open source****. Sono due cose che portano lo stesso nome: quello libero è* ***recordly.dev****.*

La logica d'uso è sempre la stessa, in qualunque registratore:

1. si sceglie **cosa** registrare (tutto lo schermo, una finestra, una zona);
2. si sceglie se registrare anche **l'audio** (microfono, suono del sistema, o entrambi);
3. si preme registra, si fa la cosa da mostrare, si preme stop;
4. si esporta in **MP4**.

> *La registrazione va fatta* ***muta e senza fretta****. La voce si aggiunge dopo, e il montaggio la sincronizza: è molto più semplice che parlare mentre si clicca.*

Le regole che fanno la differenza

- **Chiudi tutto il resto.** Una notifica che compare a metà registrazione rovina il video, e se contiene un nome o un messaggio privato è anche un problema di riservatezza. Non è un consiglio di stile: è la differenza fra un video pubblicabile e uno da rifare.
- **Registra in una risoluzione che conosci.** Se lo schermo è più piccolo del formato di destinazione (per esempio 1536×864 quando il video deve essere 1920×1080), il video andrà ingrandito e perderà nitidezza. Meglio saperlo prima.
- **Muovi il mouse con calma.** Il puntatore che sfreccia è illeggibile. Le pause di un secondo, prima e dopo ogni clic, sono quello che rende un tutorial seguibile.
- **Un video muto si può sempre montare.** Un video senza la scena che serviva, no: si rifà.

3 · Playwright: far muovere il browser da solo

Playwright è uno strumento di Microsoft, **libero e gratuito**, che comanda un browser vero — Chrome, Edge, Firefox — dall'interno di un programma. Non serve a registrare: serve a **fare le cose al posto tuo**, in modo ripetibile.

È la differenza fra registrare a mano una procedura e **descriverla una volta** perché poi si esegua da sola.

Cosa permette

- Aprire una pagina e **leggere la sua struttura**: quali elementi si possono cliccare, che testo hanno, dove sono.
- **Muovere il mouse e cliccare** su un elemento preciso, cercandolo per testo o per indirizzo.
- **Scorrere**, compilare, attendere che una pagina sia pronta.
- **Registrare** quello che succede, oppure lasciare che a registrare sia ffmpeg (vedi oltre).
- **Eseguire la stessa procedura quante volte serve**, identica.

Il punto che cambia tutto: la registrazione dallo schermo

Playwright sa produrre da sé un video della pagina, ma è uno *screencast*: **il puntatore del mouse non c'è**, perché il video viene composto dalla pagina e non dallo schermo. Il cursore va quindi disegnato dentro la pagina, con un po' di codice.

C'è però una strada migliore, e sta nel combinare due strumenti:

> ***Playwright pilota, ffmpeg registra.*** *Si apre il browser **visibile**, Playwright muove il mouse vero, e ffmpeg riprende la finestra con* `-draw_mouse 1`*. Risultato: si vedono **il puntatore vero** e **la barra degli indirizzi**, e resta tutto deterministico — perché a decidere le azioni è uno script, non una mano.*

Le due accortezze, entrambe imparate sbagliando:

- **Si registra quello che si vede.** Se la finestra del browser non è davanti a tutte le altre, nel video finisce un'altra applicazione. Prima di registrare va portata in primo piano e **verificato** che ci sia davvero.
- **Le coordinate sono in pixel fisici.** Con lo schermo al 125% di ingrandimento (1920×1080 fisici che diventano 1536×864 logici), un programma che legge le coordinate "logiche" sbaglia la zona da riprendere. Va detto al programma di leggere in pixel fisici.

Un avvertimento onesto

Playwright si presenta ai siti come **browser automatizzato**, e alcuni servizi — Google in testa — rispondono *«questo browser potrebbe non essere sicuro»* **prima ancora di chiedere le credenziali**. Non è un blocco sulla password: non c'è nessun campo dove digitarla.

La soluzione esiste, ed è il capitolo 8. Qui basti sapere che il problema non è la password.

4 · GSAP: animazione per il web e per il video

GSAP è una libreria di animazione per il web. Nasce per far muovere elementi dentro una pagina, ma il suo uso più interessante per chi fa video è un altro: **animare una pagina HTML e poi riprenderla come fotogrammi di un video**.

Serve a: far comparire un titolo con una dissolvenza, far scorrere una scheda, far muovere un elemento al momento giusto, sincronizzare il testo con la voce.

Il vantaggio rispetto a un programma di montaggio è che **tutto è descritto in un file di testo**: la stessa animazione si può rigenerare identica domani, o su un altro computer, o con un testo diverso. Un video fatto così non è un file da ritoccare a mano: è un **progetto** che si riapre.

> ***Sulla licenza, il dato verificato.*** *Dal 2025, dopo l'acquisizione da parte di Webflow, GSAP è* ***gratuito anche per uso commerciale****, e sono diventati gratuiti anche i plugin che prima richiedevano l'abbonamento (SplitText, MorphSVG, ScrollSmoother e gli altri). Attenzione però a una distinzione che si fa spesso a rovescio:* ***gratuito non vuol dire open source****. GSAP ha una licenza* ***proprietaria*** *(la "GSAP Standard License"), non MIT né Apache: si può usare gratis, anche per lavoro, ma il codice non è aperto e non si possono rimuovere le note di copyright. Per un progetto dove conta la libertà della licenza, questo è un punto da tenere presente.*

5 · Meshy: oggetti 3D

Meshy genera **modelli tridimensionali** a partire da una descrizione testuale o da una fotografia. Si ottiene un oggetto 3D — una sedia, un pianeta, un ingranaggio — che si può girare, inserire in una scena, esportare come file `.glb` o `.obj`.

A cosa serve in un video? A tutto ciò che non si può filmare: un oggetto che non esiste, una ricostruzione, un'animazione tecnica. Molti strumenti di montaggio e di animazione 3D importano questi file e li fanno ruotare davanti alla telecamera.

Meshy è un servizio **nel cloud** (nessuna scheda grafica richiesta sul proprio computer: serve un browser e un account). Il piano gratuito dà **100 crediti al mese**, circa **dieci oggetti**, un compito alla volta, e permette di scaricare **al massimo dieci modelli al mese**. I piani a pagamento partono da circa **20 dollari al mese**.

Ma il dettaglio che conta davvero è un altro, ed è quello che si scopre sempre troppo tardi:

> ***Con il piano gratuito i modelli si scaricano con licenza CC BY 4.0****: si possono usare anche commercialmente,* ***ma citando la fonte****. Con i piani a pagamento la licenza diventa "privata", cioè uso commerciale pieno* ***senza obbligo di attribuzione****.*

Tradotto: il piano gratuito va benissimo per imparare e per progetti dove puoi citare Meshy; se il modello deve finire in un lavoro dove l'attribuzione non è possibile, serve il piano a pagamento. Non è un limite tecnico, è un limite di licenza — ed è meglio saperlo prima di costruirci sopra.

6 · Comfy.org e ComfyUI: immagini e video generati

Qui conviene fare subito una distinzione, perché è la fonte di quasi tutti gli equivoci.

ComfyUI è un programma **libero e gratuito**: si installa sul proprio computer e non manda niente a nessuno. È un'interfaccia a nodi — si collegano dei riquadri fra loro e si costruisce un procedimento — per generare immagini e video con i modelli di diffusione.

Comfy Cloud è il servizio **a pagamento** con lo stesso motore, che gira sui server di chi lo fornisce: si paga un abbonamento mensile con un sistema a crediti.

	ComfyUI (locale)	Comfy Cloud
costo	**gratuito**, licenza GPL-3.0	prova gratuita, poi abbonamento
dove gira	sul tuo computer	sui loro server
cosa serve	**una scheda grafica** (vedi sotto)	solo un browser e un account
riservatezza	tutto in casa	le immagini passano dal loro servizio

Il cloud parte con **cinque esecuzioni gratuite** su schede grafiche vere e un po' di crediti in omaggio, poi gli abbonamenti vanno da circa **20 dollari al mese** (35, 100 e oltre per i piani superiori), con un sistema a crediti: un video di cinque secondi costa, indicativamente, una decina di crediti. Chi vuole solo l'API paga a ore di scheda grafica, da circa 3,50 a 11 dollari l'ora.

Il punto pratico, però, è la scheda grafica — e qui serve precisione, perché si sente dire "ComfyUI richiede una GPU" e non è esattamente così:

- La versione locale **parte anche senza scheda grafica**, usando solo il processore. È supportato ufficialmente. Ma su una macchina come questa (un i5 senza GPU dedicata) i tempi diventano **inutilizzabili**: si parla di ore per un'immagine, non di minuti.
- Quindi: "serve una GPU" è vero **per le prestazioni**, non in assoluto. Chi non ce l'ha e vuole usare ComfyUI davvero, oggi sceglie il cloud.

7 · AnyMP4 / AnyRec: conversione e montaggio

Sono programmi con l'interfaccia grafica per convertire video, registrarne lo schermo, scaricare da internet, montare. Fanno il loro lavoro, e a molti risultano più comodi di una riga di comando.

Ma qui bisogna essere chiari, perché è esattamente il tipo di strumento che in un elenco "gratuito" non dovrebbe stare senza avvertenza:

> ***Sono prodotti commerciali**, e i numeri lo dicono: **AnyMP4** costa intorno ai 12 dollari al mese (o circa 100 per una licenza valida su cinque computer), **AnyRec** circa 14,50 al mese oppure 55 per una licenza a vita. Il programma da tavolo gratuito è una **versione di prova**: serve a provare, non a produrre.*

C'è però una parte gratuita che vale la pena conoscere: entrambi offrono un **registratore di schermo *online*, dentro il browser**, senza installare niente e senza account. Le differenze contano:

	registratore online	limiti
AnyMP4	gratuito, dichiarato **senza filigrana**	massimo 24 fotogrammi al secondo (contro 60), esporta solo in WMV e MP4, meno strumenti, niente registrazione programmata
AnyRec	gratuito, senza limite di durata, senza account	**aggiunge una filigrana** sull'esportazione

Quindi, se serve una registrazione veloce e senza installare nulla, **quello di AnyMP4 è la scelta migliore** — purché 24 fotogrammi al secondo bastino (per un tutorial bastano; per un videogioco no).

Perché allora stanno in questa guida? Perché sono un **punto di partenza comodo** per chi non ha mai toccato ffmpeg: si impara cosa serve fare (convertire, tagliare, unire) con i pulsanti, e poi — quando si capisce che la stessa cosa si fa con una riga — si passa a ffmpeg, che è gratis per sempre e non ha filigrane.

Il criterio, che vale in generale

Quando un programma gratuito chiede di pagare, guarda **cosa** ti sta vendendo:

- se ti vende **l'interfaccia** (pulsanti comodi attorno a un motore gratuito), allora lo stesso lavoro si può fare con lo strumento libero e un po' di pratica;
- se ti vende **il motore** (un modello di intelligenza artificiale che gira sui loro server), allora lì il pagamento è reale, e conviene valutare quanto lo usi.

8 · Pollinations: contenuti generati via API

Pollinations è un servizio che genera **immagini, testo, audio e video** con un'interfaccia molto semplice: spesso basta un indirizzo web. È la via più diretta per aggiungere un'immagine o una voce a un progetto senza installare nulla.

Cosa funziona senza chiave

Questo è il punto interessante, ed è **verificato**:

- **le immagini si generano senza chiave**: basta aprire un indirizzo con la descrizione dentro, e si riceve un'immagine;
- **il catalogo dei modelli è pubblico**: si può leggere quali modelli esistono (sono centinaia), quanto costano e se sono gratuiti;
- **il testo dipende dall'indirizzo che usi**, e questa è la sfumatura che ho dovuto misurare per capirla:
- l'**indirizzo semplice** (`text.pollinations.ai/…`) risponde **anche senza chiave**;

- l'**API vera** (`/v1/chat/completions`), quella che si usa dal proprio programma, **pretende una chiave** e senza risponde "rifiutato".

Non si può quindi dire "è gratis" in blocco: dipende da **cosa chiedi** e da **come lo chiedi**. La regola è controllare il catalogo — che è pubblico apposta — invece di fidarsi di un'etichetta.

> *Un'ultima distinzione, che vale per tutto questo capitolo: il **codice** di Pollinations è aperto (licenza MIT), ma il **servizio** è a crediti. Sono due cose diverse: puoi leggere e riusare il programma, non il servizio che qualcun altro paga.*

La chiave senza farla passare da nessuna parte

C'è un modo per ottenere una chiave **senza incollarla in una chat, in un comando o in un file di configurazione**, e si chiama **device flow**: il servizio mostra un indirizzo e un codice, **l'utente apre quell'indirizzo nel proprio browser e approva**, e il programma riceve la chiave da solo. È lo stesso meccanismo con cui si autorizza un televisore a usare un account di streaming.

Il vantaggio è che **la credenziale non passa mai da chi la deve usare**: nessuno la scrive, nessuno la legge, nessuno la salva in un posto sbagliato.

9 · La tecnica: far entrare un agente nei propri profili

Questo è il capitolo per cui vale la pena leggere la guida, perché cambia il modo di lavorare più di qualunque strumento dell'elenco.

Il problema

Molte cose che vorremmo far fare a un assistente automatico — aprire un servizio, guardare un'interfaccia, esportare dei dati, registrare una procedura — richiedono un **accesso**: un nome utente e una password, o una sessione già aperta.

La soluzione ingenua è dare all'automatismo le credenziali. **È sbagliata**, per tre motivi:

1. la password finisce in un programma, in un file, o in un registro di log;
2. non funziona con i servizi che richiedono un secondo fattore di sicurezza;
3. e nei casi peggiori il servizio **blocca** l'accesso, perché riconosce che il browser non è guidato da una persona.

La tecnica, in tre passi

Si chiama **accesso delegato**: l'automatismo non ha le credenziali, e non gli servono.

1. **L'automatismo apre una finestra di navigazione propria** — con un profilo dedicato, mai il profilo personale di chi lo usa — e si presenta al sito come un browser normale.
2. **La persona fa l'accesso in quella finestra**, una volta sola, come farebbe sempre. La password non passa da nessun'altra parte: resta fra la persona e il sito.

3. **L'automatismo si accorge da solo che l'accesso è stato fatto**, salva la sessione e **riprende da dove era rimasto**. Da lì in avanti entra già autenticato, senza chiedere più niente.

È **la stessa idea del device flow** del capitolo precedente, applicata non a una chiave ma a una sessione di navigazione. Il principio comune è: *chi ha le credenziali le tiene, chi deve usarle riceve solo il permesso*.

I tre dettagli che fanno la differenza

Questi sono i punti in cui la tecnica funziona o non funziona, e li ho verificati uno per uno.

Il browser non deve dichiararsi automatizzato

Di norma gli strumenti di automazione si annunciano come tali, e i servizi più attenti rispondono *«questo browser potrebbe non essere sicuro»* **prima** di mostrare il modulo di accesso. Non è un blocco sulla password: è un blocco sul **browser**.

Togliendo quel segnale, la pagina di accesso torna normale. È una riga di configurazione, e cambia tutto: senza, il servizio non ti fa nemmeno provare.

Si registra quello che si vede

Vale per la registrazione dello schermo, e vale qui: se la finestra non è davanti alle altre, nel video — o nella procedura — finisce qualcosa d'altro. Prima di partire, la finestra va portata in primo piano **e verificato** che ci sia davvero. Il sistema non lo concede a chiunque: servono un paio di accorgimenti tecnici che ho dovuto scoprire sbagliando.

L'automatismo deve capire da solo quando ha finito

Non si può chiedere alla persona di premere "ho finito", perché è proprio quello che si voleva evitare. L'automatismo aspetta, e a un certo punto si accorge che l'accesso è avvenuto — perché la pagina non chiede più di entrare **e** perché i dati di sessione sono aumentati. Servono **due segnali insieme**, e serve una **attesa minima**: un accesso fatto da una persona non richiede tre secondi, e se si controlla troppo presto si conclude per sbaglio che è già tutto a posto.

Perché conviene, in una riga

Perché **l'automatismo non custodisce nessuna credenziale**, e questo risolve insieme il problema della sicurezza, quello dei servizi con doppio fattore, e quello dei servizi che bloccano i browser automatici. La persona resta la sola a conoscere la propria password, e l'automatismo lavora con il permesso che gli è stato dato.

La regola d'oro

Non lasciare mai un vicolo cieco. Se l'accesso automatico non riesce, il messaggio deve offrire **almeno due strade**, e almeno una deve essere sempre percorribile — per esempio: *«fai tu questa parte nel tuo browser, dove sei già dentro, e passami il risultato: il resto lo faccio lo stesso»*.

Un errore che dice soltanto "non posso" fa perdere tempo a chi lo legge, e fa sembrare rotto uno strumento che invece sta funzionando: ha semplicemente incontrato un caso che non può risolvere da solo.

Come si combinano: una filiera completa

Mettiamo insieme i pezzi, in un caso reale: **un tutorial che mostra come si usa un'applicazione**.

1. **Si decide cosa mostrare.** Una descrizione scritta di quello che il video deve far vedere. È il testo che guiderà tutto il resto.
2. **L'automatismo entra nel servizio** (capitolo 9): finestra propria, accesso fatto dalla persona, sessione salvata.
3. **Si legge l'interfaccia** con Playwright (capitolo 3): quali elementi ci sono, dove sono, cosa dicono.
4. **Si decide, un passo alla volta, cosa mostrare** — e qui sta l'accorgimento che fa la differenza: non si pianifica tutto in anticipo, perché dopo il primo clic la schermata cambia. Si procede **una tappa alla volta**, rileggendo la pagina dove si è arrivati.
5. **Si registra con ffmpeg** (capitolo 1), non con lo screencast: così si vede il puntatore vero.
6. **Si scrivono le didascalie e si genera la voce** (capitolo 8) per ogni tappa.
7. **Si monta tutto** con ffmpeg: le dissolvenze fra le scene, la musica di sfondo, l'abbassamento automatico della musica quando la voce parla.
8. **Si esporta** nel formato di destinazione — 1920×1080 per una piattaforma didattica, verticale per i social.

Il punto importante è **il numero 4**. La differenza fra un video che mostra davvero come si usa un programma e un video che si limita a sorvolarlo sta tutta lì: nel decidere **una mossa alla volta**, guardando dove si è arrivati, invece di scrivere un copione e sperare che regga.

Le due fasi, e perché servono

Un ultimo accorgimento, che sembra un dettaglio tecnico ed è invece sostanziale: **non si registra mentre si pensa**.

Se l'automatismo registra mentre decide la mossa successiva, nel video finiscono i secondi in cui sta "pensando" — e diventano minuti di pagina ferma. Si lavora quindi in **due fasi**:

1. **prima si esplora senza registrare**: si scoprono le tappe e si preparano le voci;
2. **poi si registra** il percorso già deciso, dove l'unica attesa è quella del parlato.

Misurato su un caso reale: lo stesso percorso passa da oltre tre minuti a venti secondi. La differenza non è di stile, è di usabilità: un video di tre minuti per mostrare tre cose non lo guarda nessuno.

In una pagina: cosa serve, quando

Serve a	Strumento	Gratuito?	Licenza
Tutto il montaggio, la conversione, la registrazione	**ffmpeg**	**sì**	libera (LGPL/GPL) — non esiste a pagamento
Registrare lo schermo a mano	**Recordly** (recordly.dev)	**sì**	aperta (AGPL-3.0)
Far muovere il browser da solo, e registrarlo	**Playwright** + ffmpeg	**sì**	aperta (Apache-2.0)
Animare schede e testi	**GSAP**	**sì, anche per lavoro**	**proprietaria**, non aperta
Oggetti 3D	**Meshy**	100 crediti/mese, 10 scarichi	gratis: CC BY 4.0 (citare) · a pagamento: senza attribuzione
Immagini e video generati in locale	**ComfyUI**	**sì**	aperta (GPL-3.0) — ma serve una GPU vera
Immagini e video generati nel cloud	**Comfy Cloud**	5 prove, poi da ~20 $/mese	commerciale
Registrare lo schermo dal browser, senza installare	**AnyMP4 online**	**sì, senza filigrana**	commerciale (max 24 fps)
Registrare lo schermo dal browser (con filigrana)	**AnyRec online**	sì	commerciale
Conversione e montaggio con i pulsanti	**AnyMP4 / AnyRec desktop**	prova con limiti	a pagamento (~12-15 $/mese)
Immagini generate via indirizzo web	**Pollinations**	**sì, senza chiave**	codice aperto (MIT), servizio a crediti
Testo generato	**Pollinations**	indirizzo semplice sì, API no	come sopra
Entrare in un servizio senza dare la password	**Accesso delegato**	tecnica, non prodotto	—

Le regole che valgono sempre

1. **Distingui "gratuito" da "aperto".** Un programma può essere gratis oggi e a pagamento domani; uno aperto, con una licenza libera, non può tornare indietro. Quando puoi, scegli l'aperto.
2. **Controlla il catalogo, non l'etichetta.** "Gratis" senza specificare *cosa* è una parola vuota: le immagini possono essere gratuite e il testo no, la versione locale gratuita e quella nel cloud a pagamento.
3. **Verifica la licenza prima di usare il risultato in un lavoro.** Vale per i modelli 3D (Meshy: con il piano gratuito devi **citare la fonte**), per le librerie di animazione (GSAP è gratis ma **non è aperto**), per le immagini generate. È il dettaglio che si scopre troppo tardi, quando il lavoro è finito.
4. **La base è ffmpeg.** Se impari quello, gli altri strumenti diventano interfacce comode attorno a qualcosa che sai già fare — e puoi cambiarli senza ricominciare da capo.
5. **Mai un vicolo cieco.** Vale per il software e vale per chi lo scrive: ogni errore deve indicare una strada, non solo un muro.
6. **La credenziale resta a chi appartiene.** L'automatismo riceve un permesso, non una password. E il permesso si può togliere.